Carnegie Mellon University
Master of
Software Engineering

# Module uses view

LG Architecture Training Program

Paulo Merson

# Agenda


- Module views
  - View showing module decomposition
  - ➡ View showing module use
  - Layered style
  - MVC style
- Data model/domain model
- Hexagonal architecture
- Microkernel pattern

# Module uses view

- It shows what other modules each module uses
- This usage dependency creates coupling between modules
- Loops should be avoided

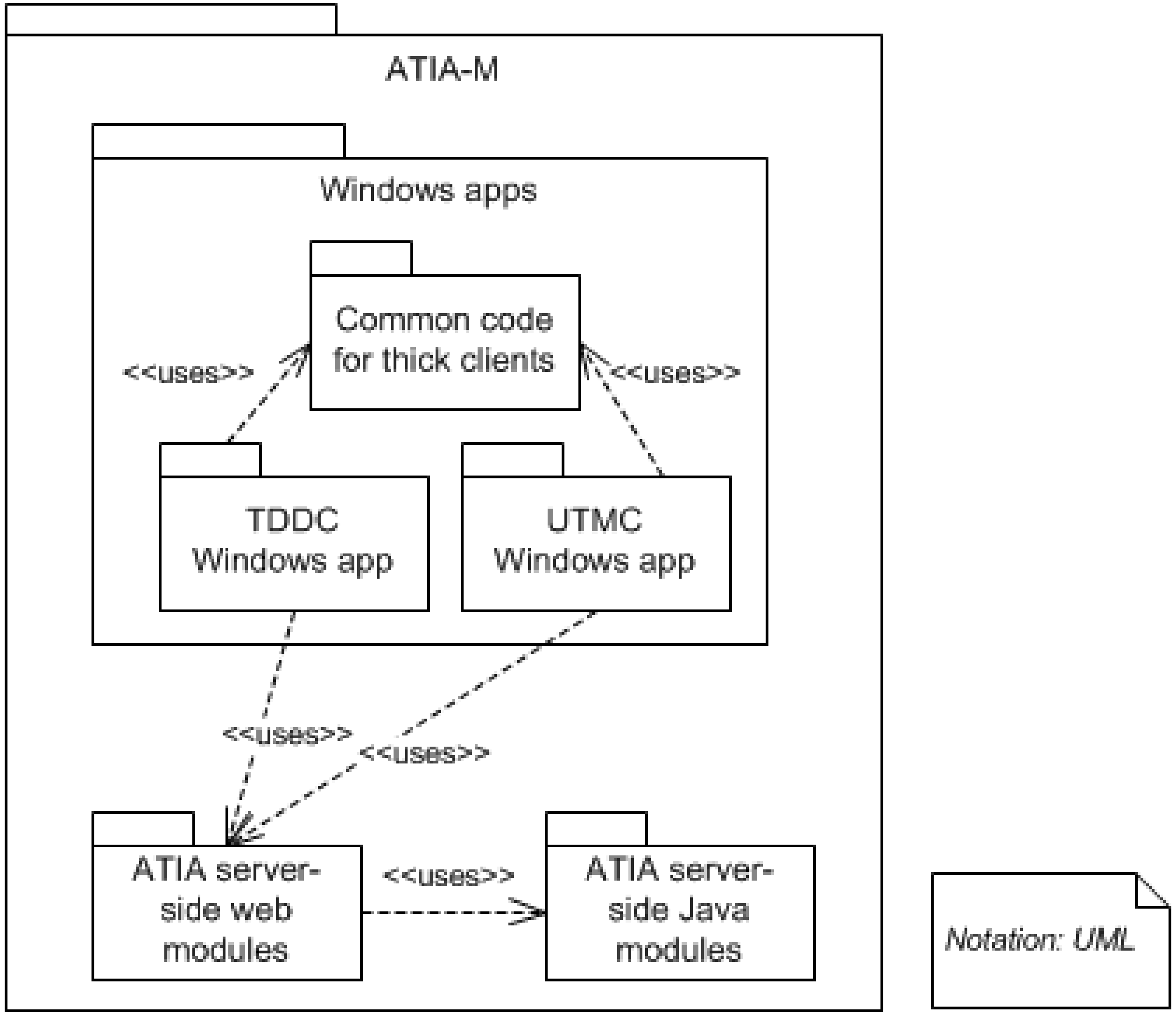
# Coupling

- Coupling refers to the degree of interdependence between software elements
  - It’s also the probability that a modification to one module will propagate to the other [Bass21]
- Removing coupling altogether is not a design goal; coupling between software modules is essential to build applications
- The goal is to limit coupling to the minimum necessary

*We want **loose coupling***
*(aka low coupling)*

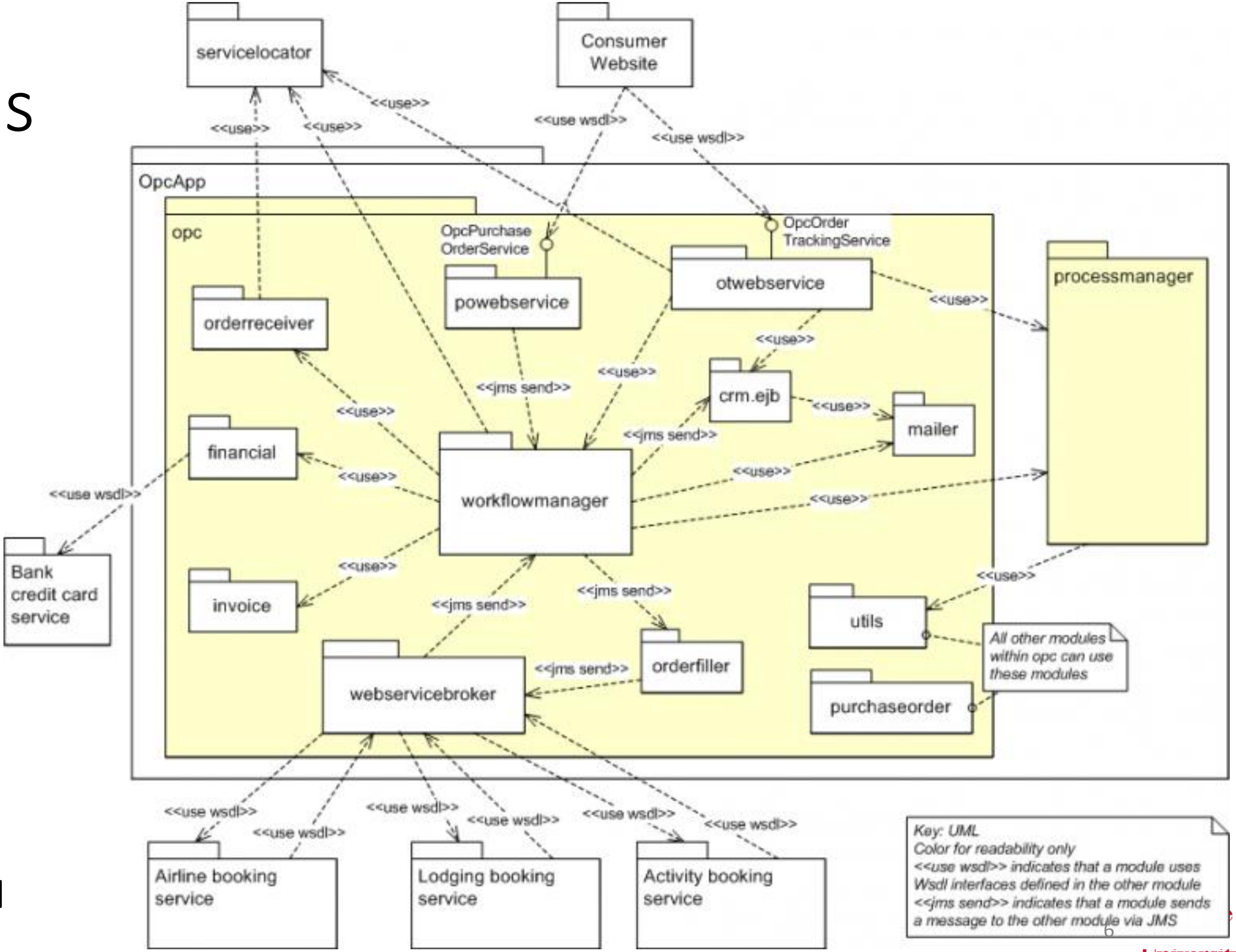
*We avoid **tight coupling***
*(aka high coupling)*

# Module uses view – example 1



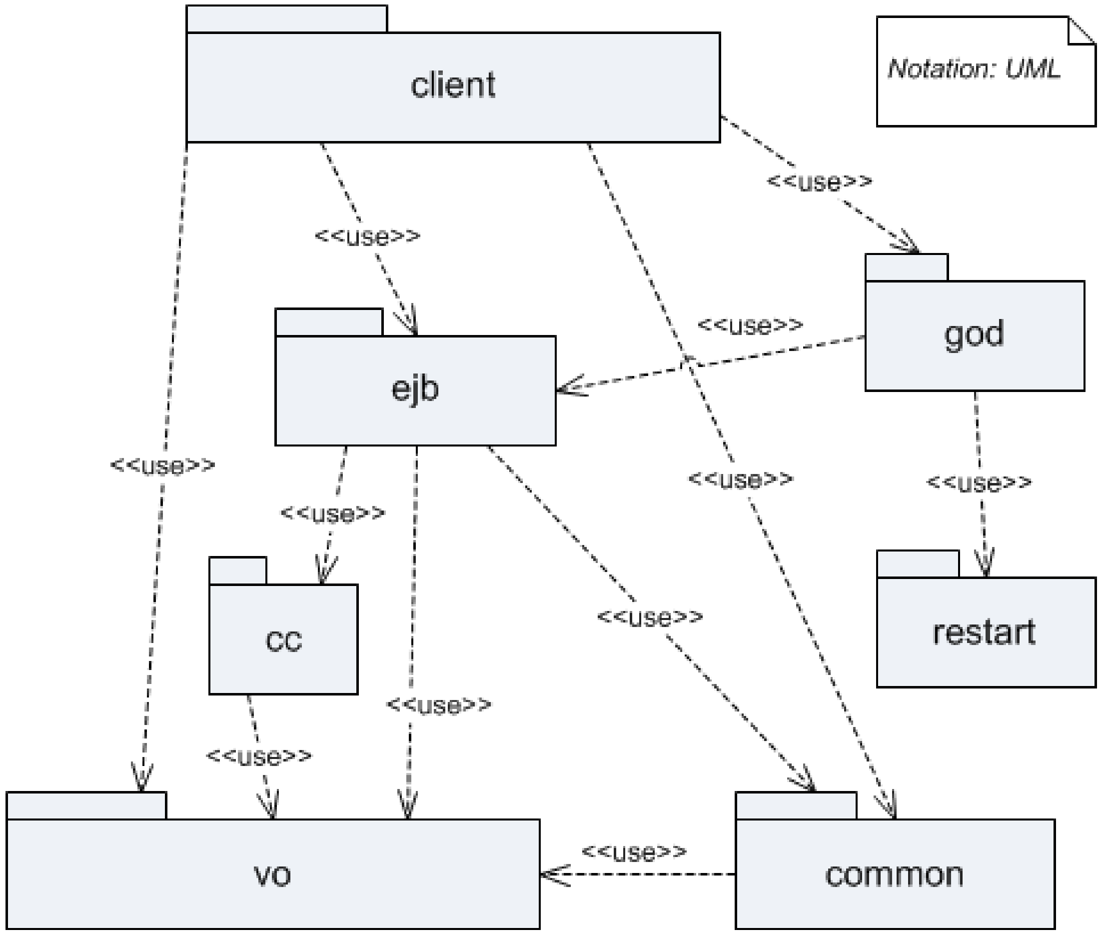

Source: [Smith03]

# Module uses view – example 2




Fonte: [Merson13a]

# Module uses view – example 3



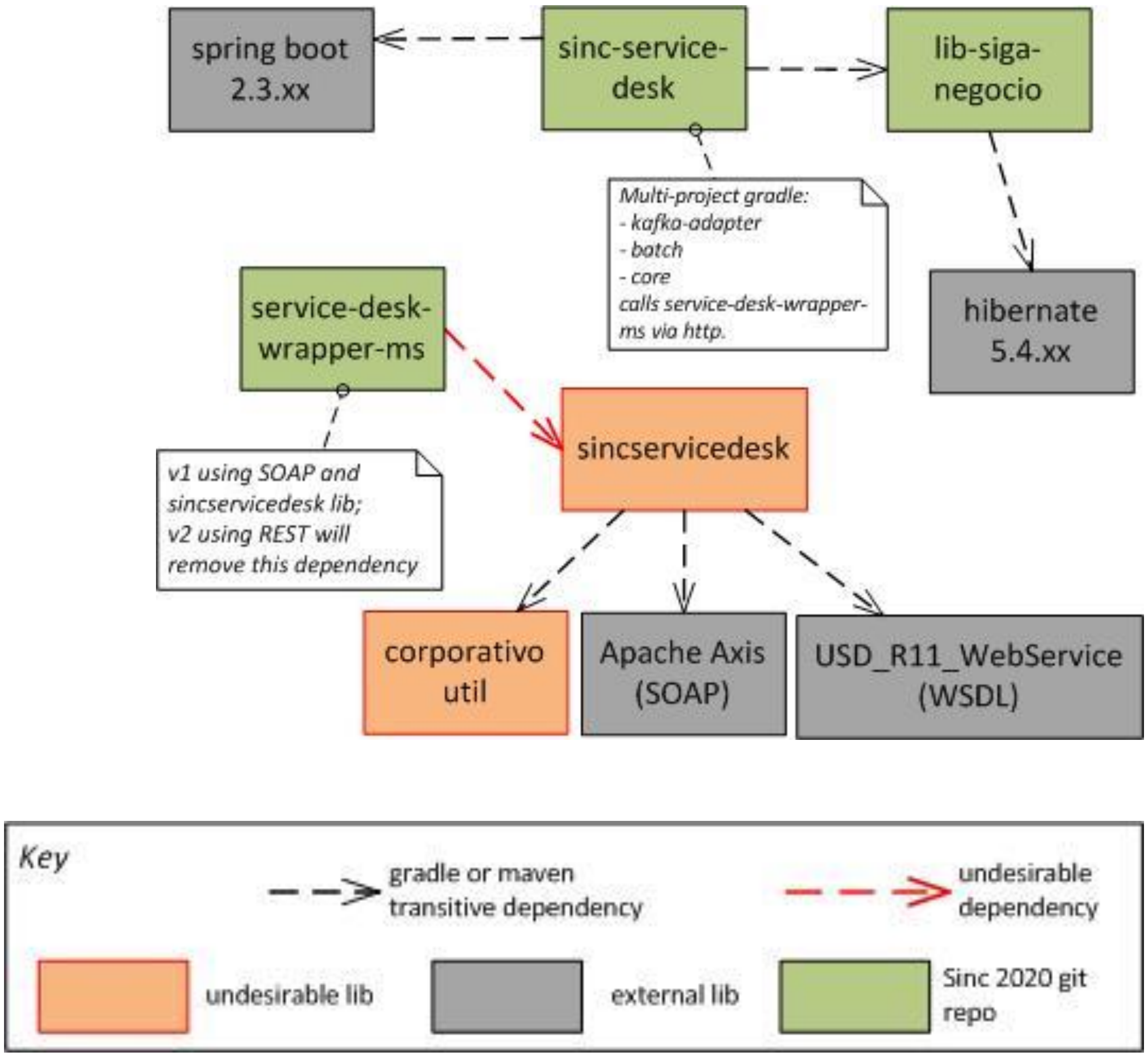

| using module / used module | client | ejb | cc | god | restart | common | vo |
|---|---|---|---|---|---|---|---|
| client | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| ejb | 1 | 0 | 0 | 1 | 0 | 0 | 0 |
| cc | 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| god | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| restart | 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| common | 1 | 1 | 0 | 0 | 0 | 0 | 0 |
| vo | 1 | 1 | 1 | 0 | 0 | 1 | 0 |

*Key: '1' means module in column uses module in row*

On the right is the Dependency Structure Matrix (DSM) of the diagram on the left

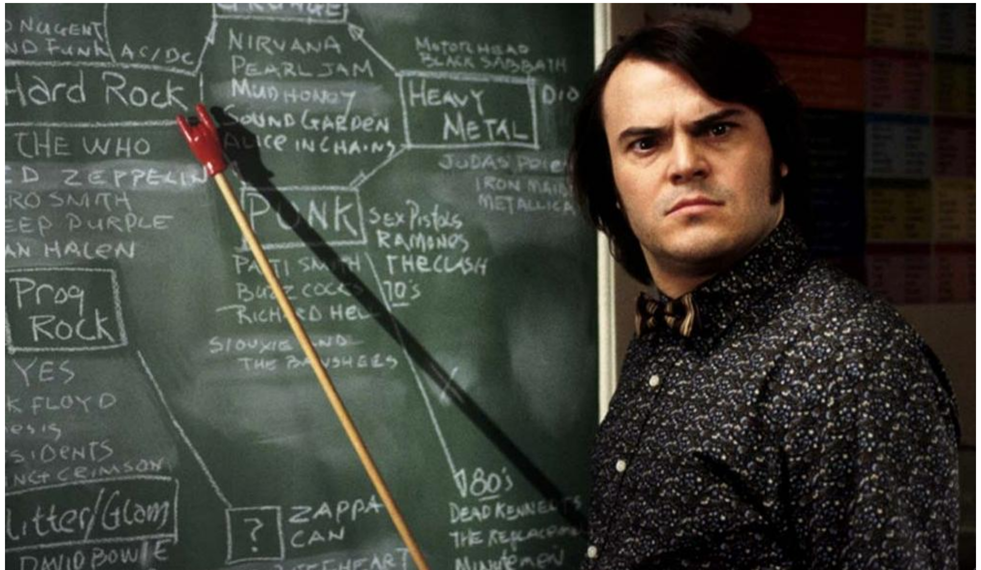
Fonte: [Clements10]

# Module uses view – example 4

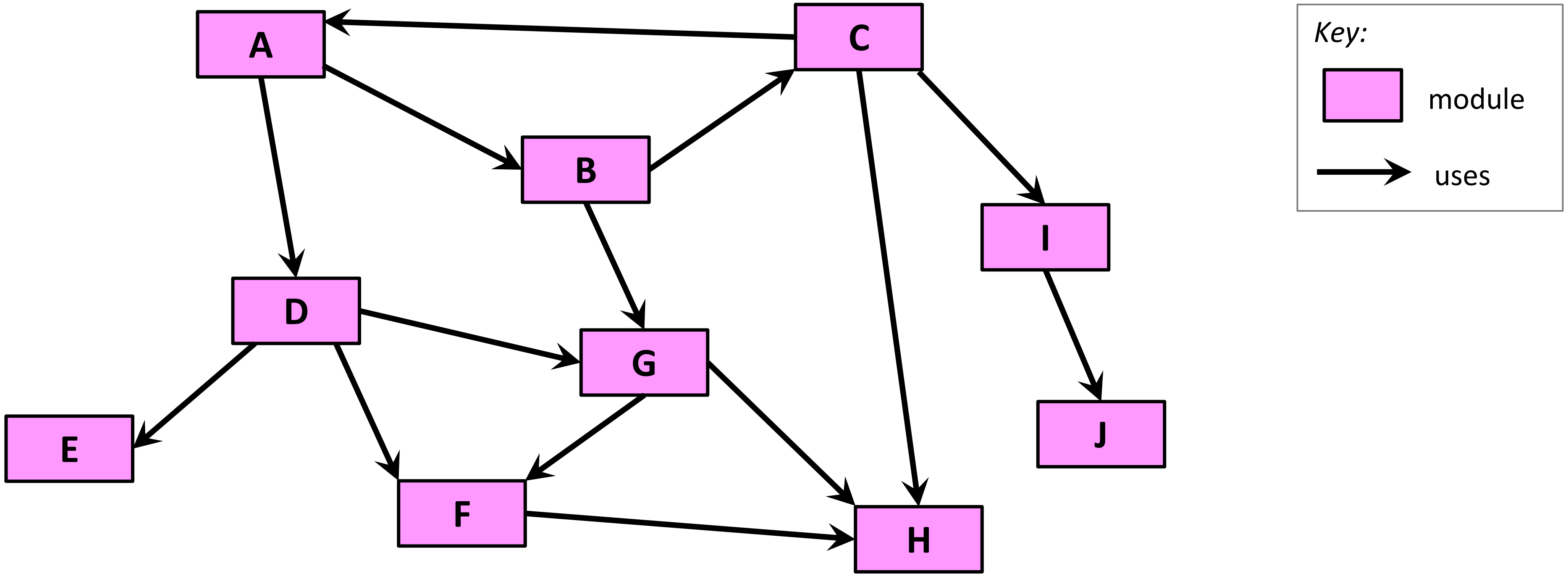


# *Uses* of the module uses view

- Onboarding of new developers
- Guide testing and debugging
- Define task allocation and project schedules
- Plan incremental development
- Impact analysis
- Guide refactorings



# Uses view is key to incremental development



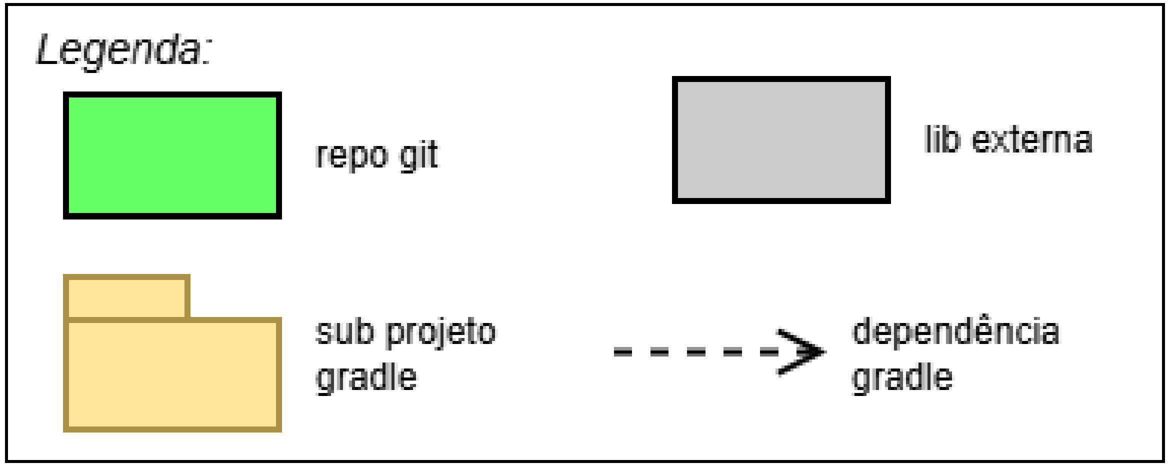

*What modules are required if I want module C in the next release?*
*And now?*

*GenAI can help you to create module uses diagrams!*

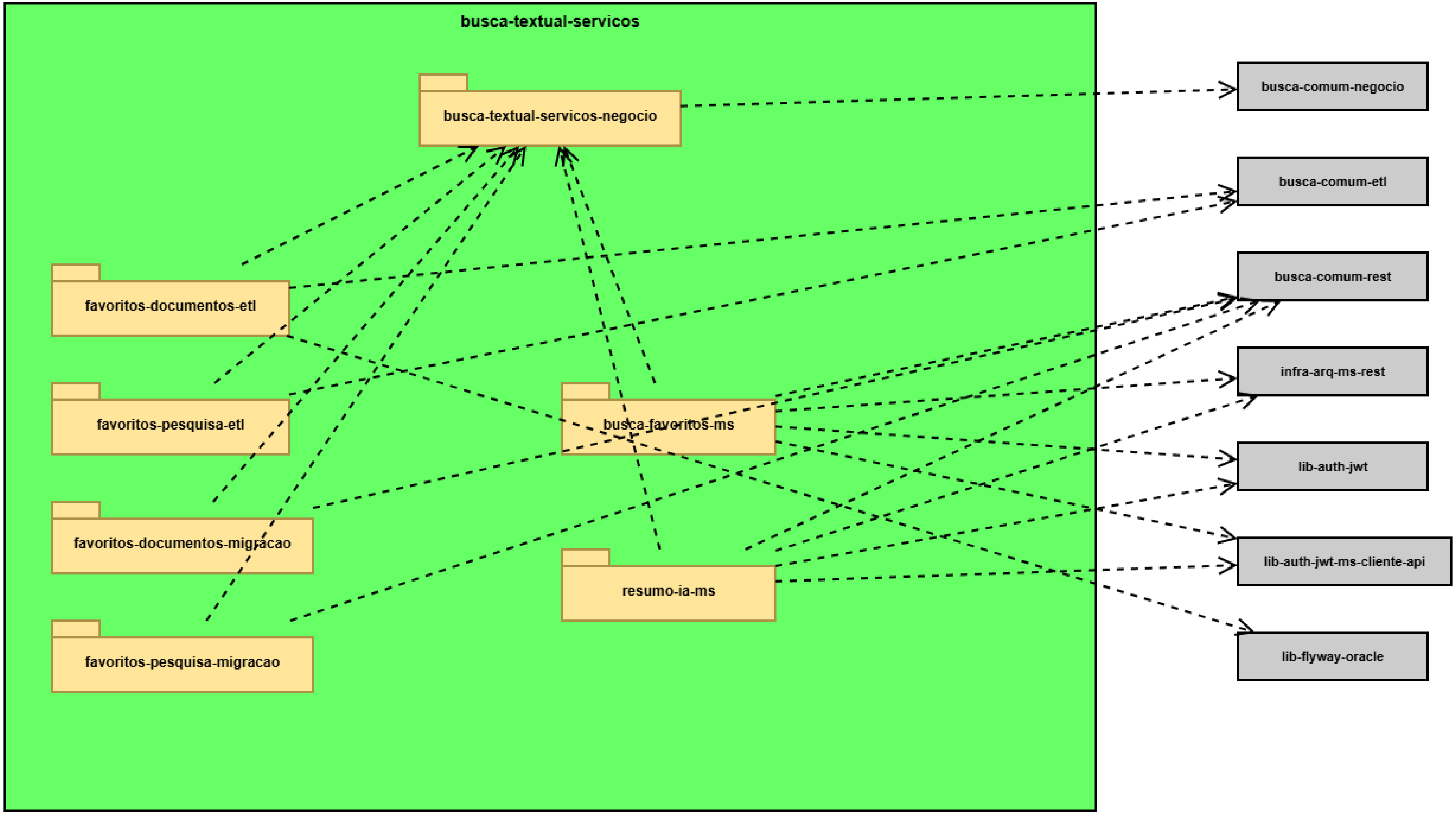
*But if the code doesn’t exist yet, it might be easier if you create the diagram!*

# Example: Claude Sonnet creating a .drawio diagram

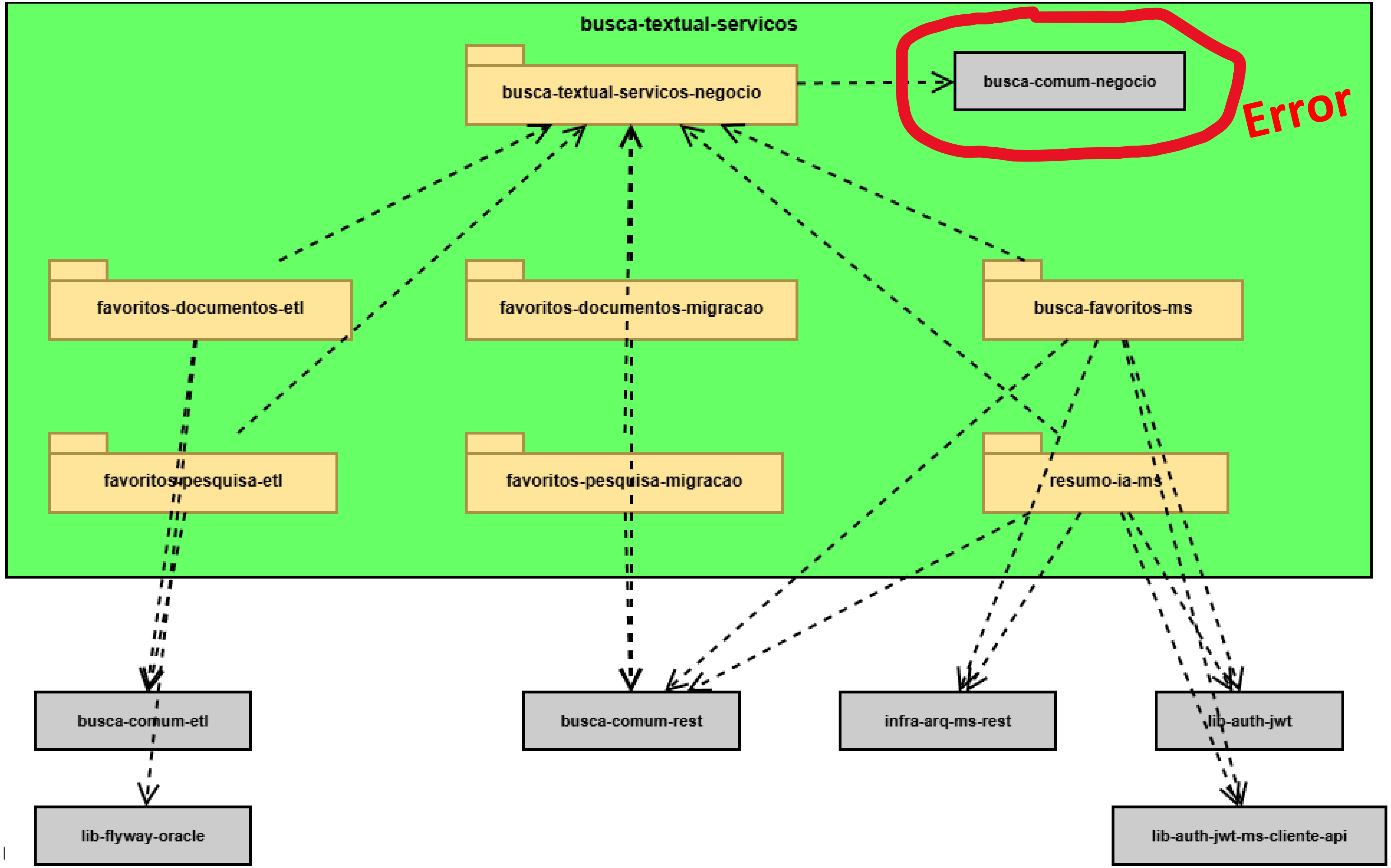
- Prompt contained
  - A very simple notation key (as a .drawio file) shown below
  - Location of a git repo
  - Instructions to map code constructs to elements in the diagram



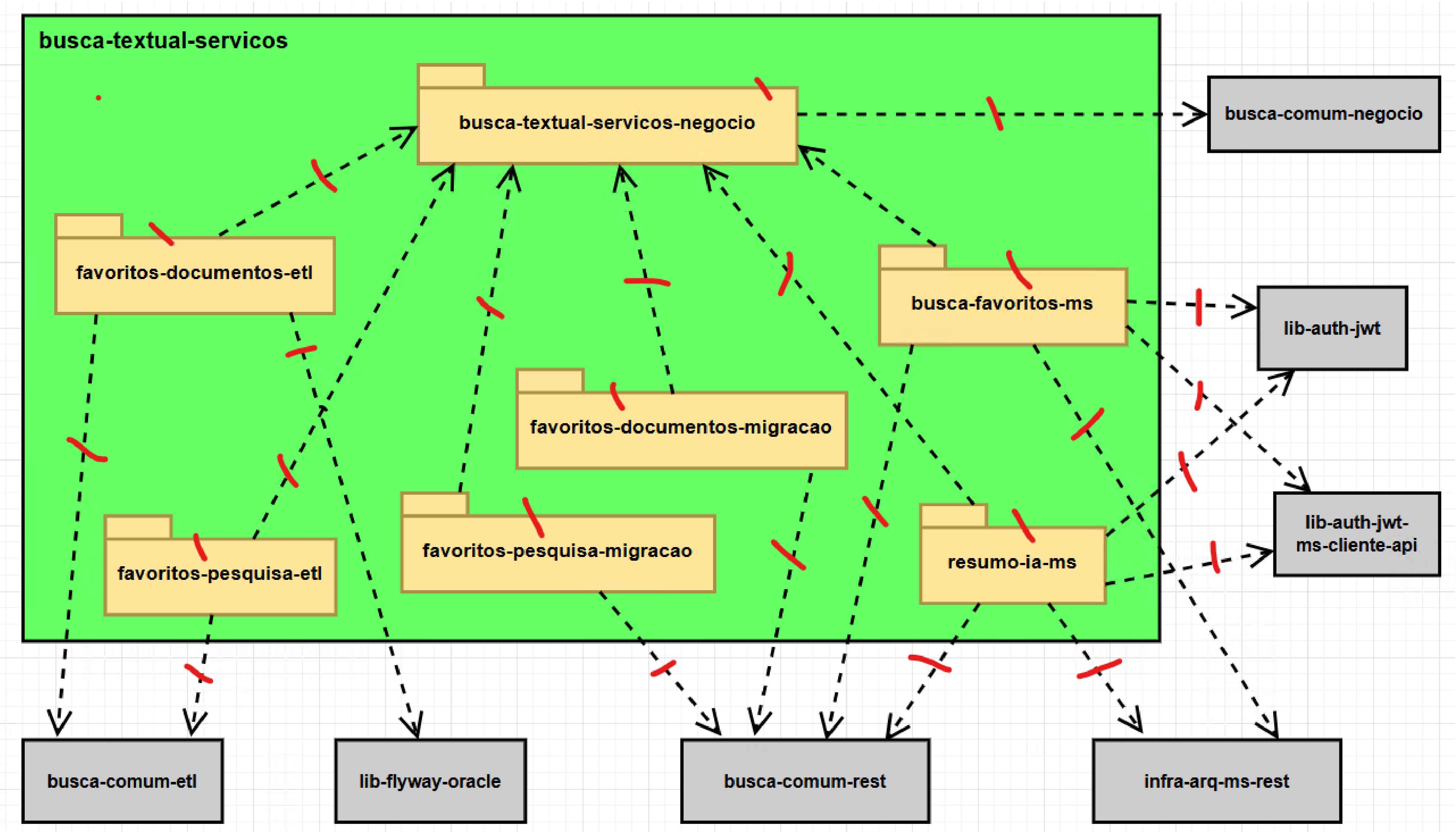
# Version 1 created by Claude Sonnet



# Version 2 after asking for a “planar graph”

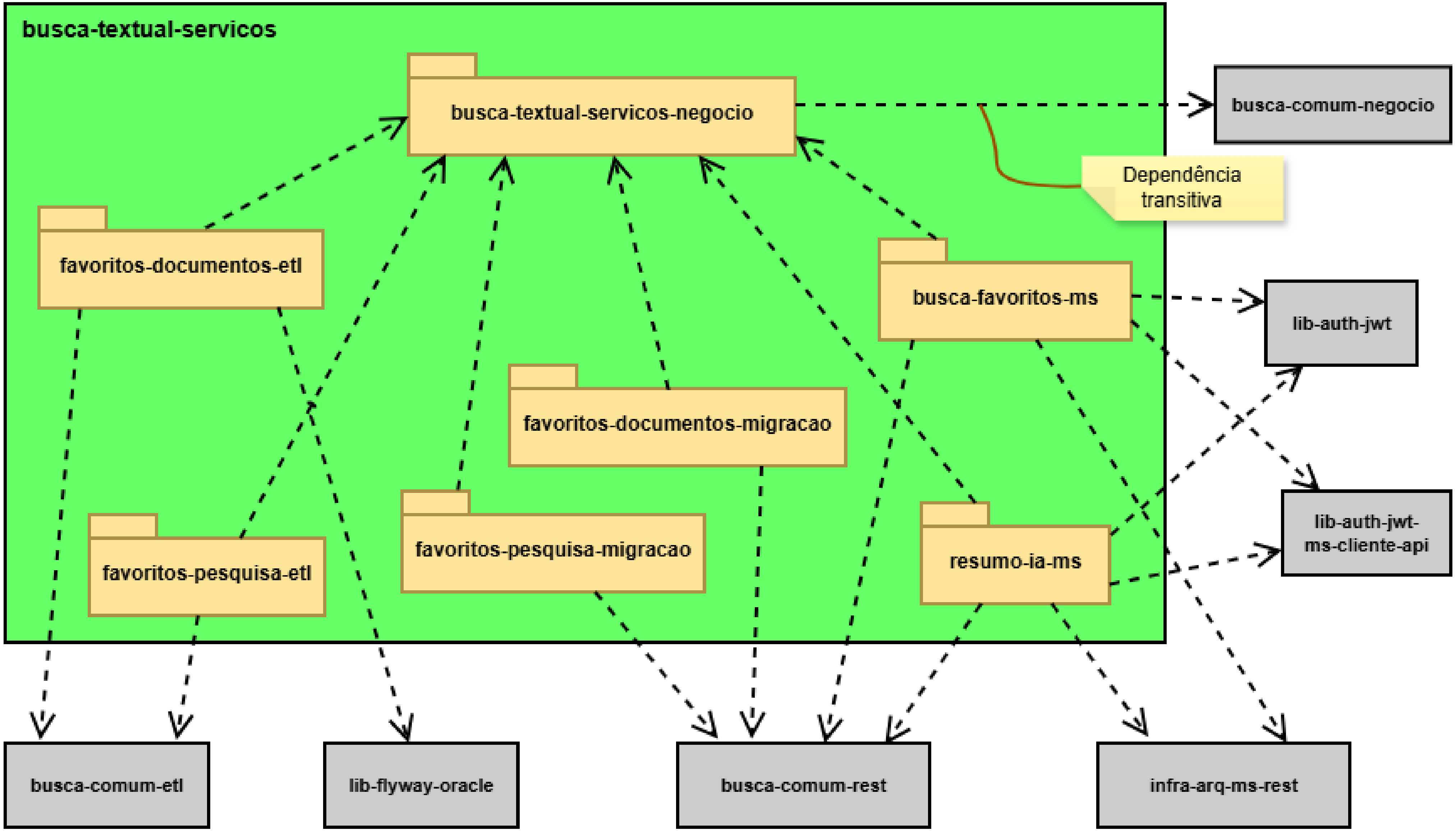


# Version 3 after manual reorg of boxes and fix




Red ticks for manual verification

# Version 4 with final manual adjustments



# Using GenAI with design diagrams

- Requires a tool that works with a machine-readable file format
- Examples of tools:
  - **Drawio** (diagrams.net) uses XML 🙂
  - **Excalidraw** uses JSON 🙂
  - **Mermaid** is text-based but limits the types of diagrams and what can be customized 😐
  - **Miro** and **Mural** don't save diagrams in text format ☹️
  - **Enterprise Architect** and **ArchiMate** can export XML but are full-fledged modeling tools 😳
- Diagram creation should allow roundtrip engineering

*Like any GenAI non-deterministic output, diagrams should be reviewed by a human*

# *Questions?*

***Paulo Merson***
***pmerson@cmu.edu***



Carnegie Mellon University